Vector, Graph and RAG Research

IS373 | Independent student guide | November 3, 2026

Everyone researches vector databases, graph databases, retrieval-augmented generation (RAG), chunking and content import. Your individual hosted chat may use vector retrieval, graph retrieval or both. Your group applies a justified architecture to one of the course content problems. Research supports the Plan review; implementation develops through Build and Release.

1. Start with a reader, question and source

Choose a small content domain and a useful question. Start with approximately five short permitted documents. Public availability does not automatically grant every reuse right. Record permission, original URL, author, date/version and stable source ID. Synthetic fixtures are acceptable when labeled.

Stage	What to understand	Evidence to save
Import	Which sources and formats enter?	Source manifest and original fixture
Prepare	How do you parse, clean and deduplicate?	Before/after text and import outcome
Represent	Chunks/vectors, entities/relationships or both?	Sample representation with source IDs
Retrieve	Which evidence answers the question?	Query, returned IDs/passages or paths
Generate	How does retrieved evidence enter model context?	Actual input context and resulting answer
Review/deliver	Can a reader check the answer and sources?	Visible references, limits and feedback

Research the concepts before choosing tools. For each important claim record its source, access date and relevant section. Separate documentation facts, your hypotheses and actual observations.

Copyable prompt: scope the research

Help me investigate RAG for [reader/task] using [permitted content]. Ask about questions, source permission and constraints. Explain import, cleaning, chunking, embeddings, entities/relationships, retrieval and generation. Compare keyword/SQL, vector and graph approaches. Cite current primary documentation. Do not invent sources, numeric embeddings, benchmark results or test runs.

2. Compare the retrieval approaches

Approach	What it represents or matches	Useful question	Important limit
Keyword/structured baseline	Query terms or explicit fields	Which document contains this policy code?	Paraphrases may need additional handling
Vector	Numeric embeddings of text and similarity	Which passages discuss welcoming new members?	Similarity alone does not establish truth or relationships
Graph	Entities, typed relationships and properties	Which committee owns the policy cited by this guide?	Missing or wrong relationships distort retrieval
Combined	Complementary retrieval signals/context	Find relevant passages, then follow a supported relationship	Added complexity needs a demonstrated benefit

PostgreSQL documents full-text retrieval; Qdrant documents vector search and metadata filtering; Neo4j documents graph structures and also supports vector indexes. Database brands and retrieval approaches are not mutually exclusive categories. [Technical sources are on the final page.]

Explain both representations

Vector example: document ID, chunk ID, text, source location, metadata, embedding model/version and real vector. Embed the question compatibly with the stored content. An embedding is a representation, not a summary or factual verification.

Graph example: Document A CITES Policy B; Committee C OWNS Policy B. Define node labels, relationship types, properties and supporting source locations. Check extracted entities and relationships. Label inferred relationships rather than presenting them as stated facts.

Make a justified choice

Write: our questions; selected vector/graph/both approach; why it fits; what the simpler baseline can do; expected benefit; how we will check it; hosting/cost limits; and a smaller fallback. Research both even if you implement only one.

Copyable prompt: review the choice

Using my source descriptions, data model and questions, compare vector, graph and a simple baseline. Identify missing source provenance and assumptions. Recommend a bounded experiment for each plausible option. Separate documented capabilities from unexecuted claims about our project. Ask me to explain why I chose the architecture.

3. Investigate chunking and content import

Chunking splits source text into retrievable units. Boundaries may follow length, paragraphs, headings or document structure. Large chunks can carry more context but include irrelevant material; small chunks can lose context. Overlap can preserve continuity but repeat content. These are trade-offs to evaluate on your own questions, not a universal best size.

Use one short document and compare two chunking choices. For a simple experiment, compare paragraph/heading boundaries with a fixed-length split, and record the unit as characters or tokens. Do not confuse the two. Keep the model/tokenizer identifier if you measure tokens. Inspect a boundary that cuts a useful passage and explain what changes.

Source-to-index worksheet

Field	Example or decision
Source	S01, original URL, version/date, permission
Prepared text	Cleaning decisions, preserved headings and source locations
Chunk	S01-C03, text, location, boundary/overlap rule
Metadata	Title, source ID, date, section and access scope
Embedding, if used	Model/version, dimensions, configuration and compatible query embedding
Graph, if used	Entities/relationships, supporting passage and validation decision
Import behavior	Added, unchanged, updated, skipped or rejected; reason

Check repeatability and changes

Import the same source twice and inspect actual stored IDs/counts. It should not create unintended duplicates. Change or remove a source and explain how stale chunks/relationships are updated or removed. Record an invalid or missing-input case. Keep originals and source mappings so answers remain traceable after processing.

Graph-only projects still explain chunking and embeddings in research, but implement the representation and evidence units their chosen graph retrieval needs. You do not need an unused vector index simply to satisfy a diagram.

Copyable prompt: examine preparation

Here is my permitted source text and proposed chunk or graph representation: [evidence]. Identify lost context, broken boundaries, ambiguous entities and missing source locations. Help me compare two choices and design repeat-import/update checks. Do not claim the checks passed unless I provide actual results.

4. Trace a RAG answer

RAG supplies retrieved external evidence to a model for generation. The original RAG paper combined retrieval over an external document index with a generation model. In your system, keep preparation, retrieval and answer generation visible enough to inspect separately.

Preparation path: select sources, parse/clean, retain provenance, create chunks or entities/relationships, then import/index the representation.

Question path: receive the question, apply access rules, retrieve relevant evidence, select context, generate a draft, display source references, and handle insufficient evidence. For graph retrieval, explain how the question becomes a bounded query/traversal and how returned facts connect to source passages.

A small worked scenario

A fictional club has a membership policy, an events guide and a committee chart. Ask where new members can get support, which committee owns a cited policy, and when the next meeting occurs. The final question may be unanswerable because the documents omit the date. Identify expected evidence before running the system. Do not substitute a plausible invented date.

Compare five queries

Include an exact term, a paraphrase, a relationship question where relevant, an ordinary useful question and an unanswerable case. For each save:

- Expected source IDs/passages or expected missing-evidence behavior.
- Actual retrieved IDs, rank or graph paths and selected context.
- Actual generated answer and whether each important claim has support.
- Whether citations open the correct source and location.
- Observed failure, change and result of rerunning the same query.

Measure retrieval quality separately from answer quality. If useful, record how many expected relevant items appear among the first k results, defining your relevance judgments and k. Only report latency/cost when measured; record conditions. A tiny query set supports a classroom comparison, not broad performance claims.

Copyable prompt: evaluate understanding

Quiz me one question at a time about our import, chunking, embeddings, graph schema, retrieval and generation. Ask me to trace a real source into an answer and diagnose a failure. Assess results only from my supplied query outputs. Do not invent citations, tests or evidence that an answer is correct.

5. Prepare a hosted demonstration

Integrate the chat into your own hosted portfolio site. Reuse existing FastAPI, hosting and CI where suitable. Use real retrieval and generation for the working RAG demonstration. A mock can help interface development, but label it and do not claim it proves RAG behavior.

Explain the database/model choices, configuration and limits. Keep credentials on the server, database services private and import/admin actions protected by server-side permissions. Apply bounded request limits to a public demo. Treat instructions found in imported documents as source content, not authority over your application. Use permitted material and keep private student records out of the corpus.

What to bring through the cycle

Stage	Bring or retain
Nov 3/5	Current Studio progress, initial technical questions and a source/content idea
Nov 10/12	Vector/graph/RAG explanation, sources, sample representation and questions
Plan, Nov 17	Chosen approach, source manifest, import/retrieval diagram and expected evidence
Build, Nov 24	Small end-to-end chat, hosted foundation, actual import/query outputs and limits
Release, Dec 3	Working hosted service, persistence, repeatable setup, references and evaluation
Improve, Dec 10	Real peer feedback, meaningful change/retest and portfolio case study

Your group applies the same reasoning to its selected course problem and must add a useful shared workflow beyond copying an individual chat. Keep one combined individual record and one shared group record. Follow Canvas for exact instructions and submissions. Existing Nov 6 Studio scope stays unchanged.

Primary sources

- PostgreSQL: full-text search
- Qdrant: vector search
- Qdrant: filtering
- Qdrant: local quickstart
- Neo4j: graph databases
- Neo4j: vector indexes
- Lewis et al.: Retrieval-Augmented Generation, 2020

Sources checked November 3, 2026. Chunking exercises, suggested corpus/query sizes and course connections are teaching recommendations to test, not vendor guarantees. Use current primary documentation for the specific version you implement.